МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«_____»

Кафедра _____

ЛАБОРАТОРНАЯ РАБОТА № 3

по дисциплине «Основы алгоритмизации и программирование»

Эмпирический анализ временной сложности алгоритмов сортировки.
Часть 2: алгоритмы «разделяй и властвуй»
(быстрая сортировка и сортировка слиянием)

Вариант 17

Направление:	38.03.05 «Бизнес-информатика»
Курс:	2
Выполнила:	студент группы БИ 2-1
ФИО:	Осипова Анна Алексеевна
Проверил:	_____

2026

1. Цель работы

Изучить рекурсивные алгоритмы сортировки, построенные по принципу «разделяй и властвуй», экспериментально подтвердить их сложность O(n log n) и провести сравнительный анализ с квадратичными алгоритмами из лабораторной работы № 2.

2. Задание варианта 17

Параметр	Значение
Опорный элемент	Сл — случайный
Схема разбиения	Л — Ломуто
Сортировка слиянием	В — восходящая (bottom-up)
Размеры n (эксперимент 2)	30 000; 60 000; 120 000; 240 000
Дополнительное задание	Г5 — устойчивость на записях (регион, выручка)

Язык реализации: Python 3.10+. Используются функции bubble_sort, insertion_sort из ЛР № 2 и модуль sorts_common.py.

3. Теоретическая часть

3.1. Парадигма «разделяй и властвуй»

Метод предполагает три этапа: разделение задачи на подзадачи меньшего размера, рекурсивное решение подзадач (властвование) и комбинирование решений. Трудоёмкость описывается рекуррентностью $T(n) = a \cdot T(n/b) + f(n)$. При $a = b = 2$ и $f(n) = \Theta(n)$ по основной теореме $T(n) = \Theta(n \log n)$.

3.2. Сортировка слиянием (восходящая)

Восходящая (bottom-up) версия сливает сначала пары элементов, затем блоки длины 2, 4, 8 … без рекурсии. На каждом уровне выполняется $\Theta(n)$ операций, число уровней — $\log n$, поэтому $T(n) = \Theta(n \log n)$ во всех случаях. Дополнительная память — $\Theta(n)$. Использование сравнения «≤» обеспечивает устойчивость.

3.3. Быстрая сортировка (Ломуто, случайный опорный)

Схема Ломуто: опорный элемент ставится в конец подмассива, один указатель i отмечает границу «левой» части. Случайный выбор опорного даёт ожидаемое время $\Theta(n \log n)$ для любых входных данных. Худший случай по-прежнему $\Theta(n^2)$, но вероятность его пренебрежимо мала. Рекурсия выполняется только в меньшую часть — глубина стека $O(\log n)$. Сортировка неустойчива, дополнительная память $O(\log n)$.

3.4. Сводная характеристика

Алгоритм	Лучший	Средний	Худший	Память	Устойчивость
Пузырьком	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Да
Вставками	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Да
Слиянием (В)	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	Да
Быстрая (Сл, Л)	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$O(\log n)$	Нет
sorted() / Timsort	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$	Да

4. Блок-схемы алгоритмов (Mermaid)

4.1. Быстрая сортировка

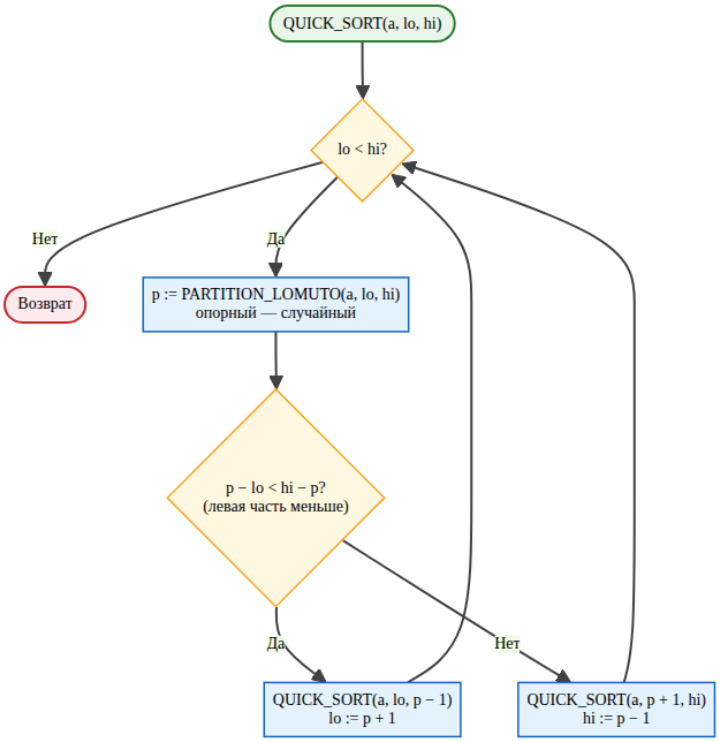


Рисунок 1 — Блок-схема быстрой сортировки (рекурсия в меньшую часть)

4.2. Разбиение Ломуто со случайным опорным

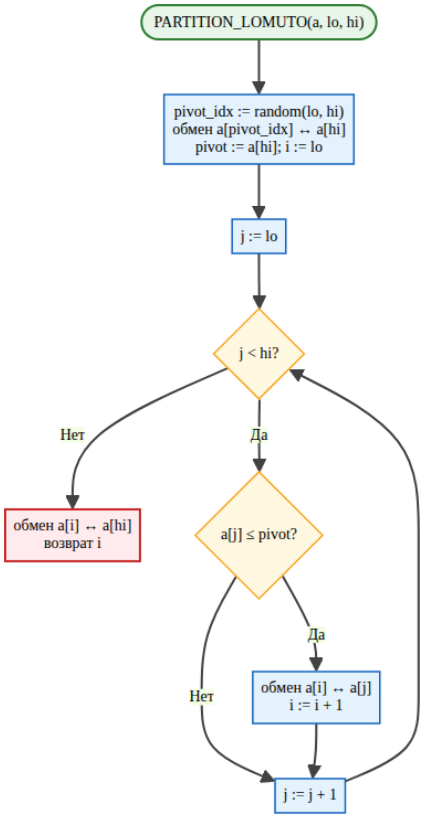


Рисунок 2 — Блок-схема PARTITION (схема Ломуто)

4.3. Восходящая сортировка слиянием

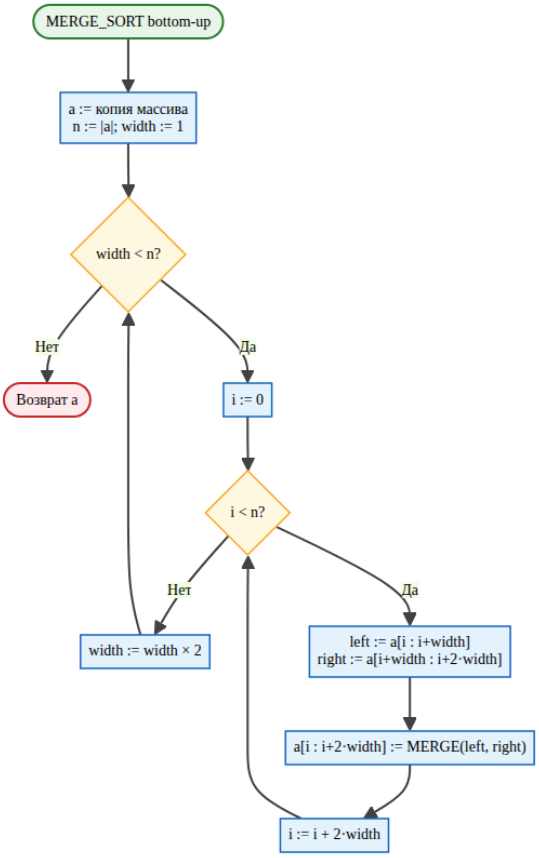


Рисунок 3 — Блок-схема bottom-up merge sort

4.4. Процедура MERGE

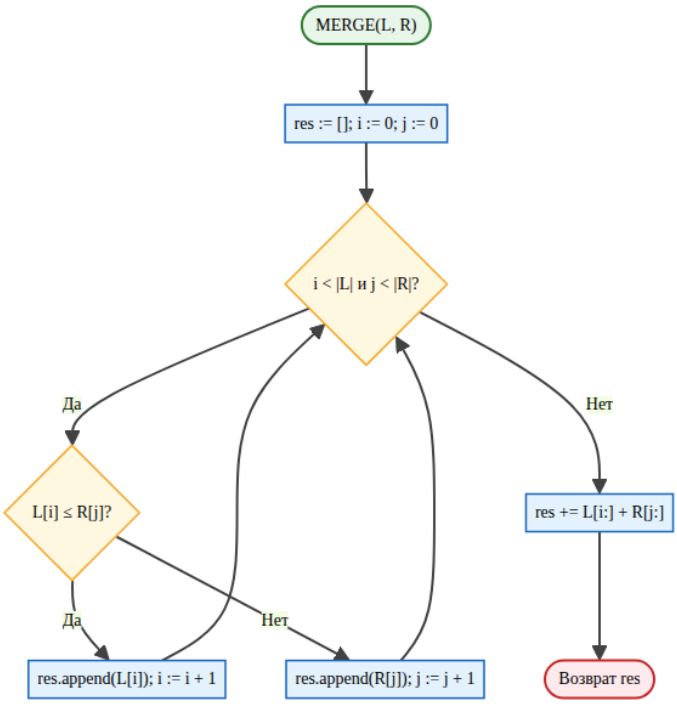


Рисунок 4 — Блок-схема слияния двух упорядоченных массивов

5. Листинг программы

5.1. Быстрая сортировка (вариант 17)

```
def quick_sort(arr):
    a = arr.copy()
    _quick_sort(a, 0, len(a) - 1)
    return a
def _quick_sort(a, lo, hi):
    while lo < hi:
        p = _partition_lomuto(a, lo, hi)
        if p - lo < hi - p:          # рекурсия в меньшую часть
            _quick_sort(a, lo, p - 1)
            lo = p + 1
        else:
            _quick_sort(a, p + 1, hi)
            hi = p - 1
def _partition_lomuto(a, lo, hi):
    pivot_idx = random.randint(lo, hi)
    a[pivot_idx], a[hi] = a[hi], a[pivot_idx]
    pivot = a[hi]
    i = lo
    for j in range(lo, hi):
        if a[j] <= pivot:
            a[i], a[j] = a[j], a[i]
            i += 1
    a[i], a[hi] = a[hi], a[i]
    return i
```

5.2. Восходящая сортировка слиянием

```
def merge_sort(arr):
    a = arr.copy()
    n = len(a)
    width = 1
    while width < n:
        for i in range(0, n, 2 * width):
            left = a[i : i + width]
            right = a[i + width : i + 2 * width]
            a[i : i + 2 * width] = _merge(left, right)
        width *= 2
    return a
def _merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:      # «<=» — устойчивость
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

6. Характеристики вычислительной системы

Измерения выполнены на удалённом стенде (Linux, Python 3.10+). Таймер — time.perf_counter(), итог — медиана из 3 повторов. Абсолютные значения зависят от конкретной системы.

7. Результаты измерений

7.1. Эксперимент 1 — четыре алгоритма (случайные данные)

n	Пузырьком, с	Вставками, с	Быстрая, с	Слиянием, с
1000	0,0334	0,0133	0,0010	0,0014
2000	0,1353	0,0547	0,0023	0,0027
3000	0,2974	0,1213	0,0033	0,0040
4000	0,5307	0,2191	0,0050	0,0057
5000	0,8537	0,3368	0,0057	0,0072

При n = 5000 быстрая сортировка быстрее пузырьковой примерно в 150 раз, вставками — примерно в 59 раз.

7.2. Эксперимент 2 — O(n log n) на больших массивах

n	Быстрая, с	Слиянием, с	sorted(), с	Слияние / Быстрая
30 000	0,0450	0,0508	0,0045	1,13
60 000	0,0886	0,1103	0,0101	1,24
120 000	0,1890	0,2435	0,0222	1,29
240 000	0,4393	0,7275	0,0570	1,66

7.3. Проверка роста T(2n)/T(n)

Переход	Быстрая	Слиянием	Теория O(n log n)	Теория O(n^2)
30 000 → 60 000	1,97	2,17	≈ 2,12	4
60 000 → 120 000	2,13	2,21	≈ 2,11	4
120 000 → 240 000	2,32	2,99	≈ 2,10	4

Отношения близки к теоретическому ≈ 2,1 для Θ(n log n) и далеки от 4 (квадратичный рост). На переходе 120k→240k у слияния отношение выше из за накладных расходов Python на создание списков при больших n.

7.4. Эксперимент 3 — структура данных (n = 50 000)

Тип данных	Быстрая, с	Слиянием, с
Случайные (R)	0,0769	0,1000
Упорядоченные (S)	0,0652	0,0598
Обратно упорядоченные (V)	0,0691	0,0572
Почти упорядоченные (N)	0,0663	0,0790
Много повторов (D)	7,1309	0,0799

На данных типа D (много одинаковых значений) схема Ломуто деградирует: почти все элементы попадают в одну часть разбиения, и время становится близким к Θ(n^2). Случайный опорный не спасает при доминировании равных ключей. Сортировка слиянием сохраняет Θ(n log n) на всех типах данных.

7.5. Графики

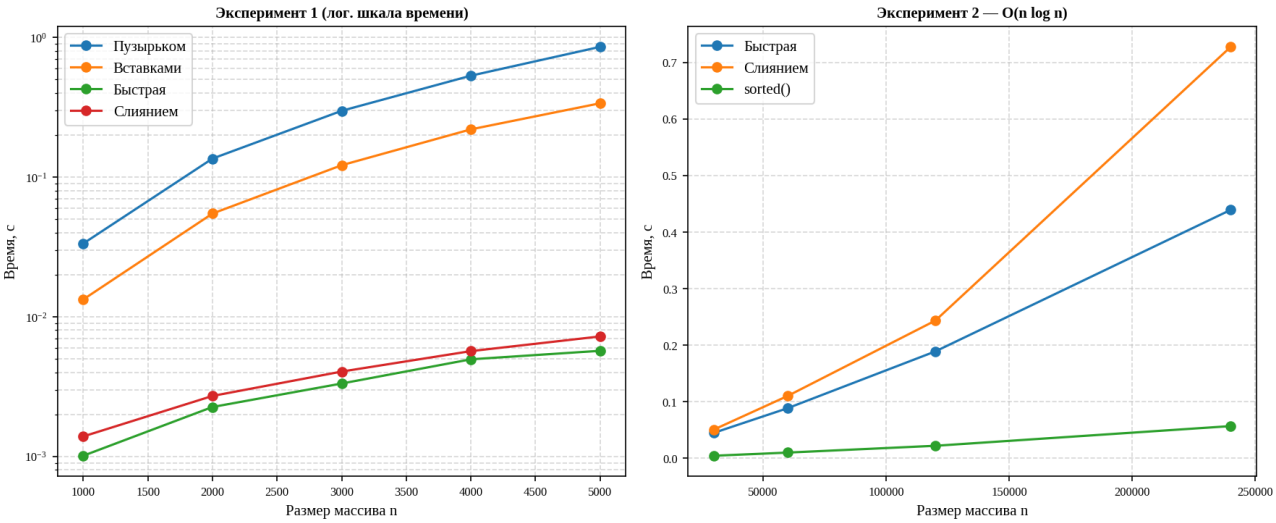


Рисунок 5 — Эксперименты 1 и 2



Рисунок 6 — Эксперимент 3: влияние структуры данных

8. Дополнительное задание Г5

Сортировка записей вида (регион, выручка) по выручке. Проверяется сохранение относительного порядка регионов с одинаковой выручкой.

Исходные	Merge sort	Quick sort
Север — 1000	Юг — 500	Юг — 500
Юг — 500	Запад — 750	Запад — 750
Восток — 1000	Север — 1000	Восток — 1000
Запад — 750	Восток — 1000	Север — 1000
Центр — 1000	Центр — 1000	Центр — 1000

Порядок записей с выручкой 1000 в исходном списке: Север → Восток → Центр. Merge sort сохранил этот порядок (устойчива). Quick sort изменил порядок (Север и Восток поменялись) — неустойчива.

Вывод: для многоуровневой сортировки отчётов (сначала по выручке, затем по региону или наоборот) пригодна только устойчивая сортировка — слияние или Timsort (sorted()). Быстрая сортировка для таких задач не подходит.

9. Сводная сравнительная таблица

Алгоритм	Ср. сложность	t (n=5000), с	Ускорение*	Память	Устойч.
Пузырьком	$\Theta(n^2)$	0,8537	1	O(1)	Да

Алгоритм	Ср. сложность	t (n=5000), с	Ускорение*	Память	Устойч.
Вставками	Θ(n²)	0,3368	2,5	O(1)	Да
Слиянием	Θ(n log n)	0,0072	119	Θ(n)	Да
Быстрая	Θ(n log n)	0,0057	150	O(log n)	Нет

* Ускорение относительно пузырьковой сортировки при n = 5000.

10. Выводы

1. Эксперимент подтвердил различие классов сложности: при n = 5000 алгоритмы O(n log n) быстрее квадратичных в 50–150 раз; на логарифмическом графике кривые квадратичных алгоритмов растут заметно круче.

2. Для быстрой сортировки и сортировки слиянием отношение T(2n)/T(n) составляет примерно 2,0–2,3, что соответствует Θ(n log n) и существенно отличается от 4.

3. Быстрая сортировка (случайный опорный, Ломуто) на случайных данных стабильно опережает восходящую сортировку слиянием; на данных с большим числом повторов схема Ломуто деградирует — это известное ограничение, устраняемое трёхпутевым разбиением.

4. Случайный опорный элемент защищает быструю сортировку от вырождения на упорядоченных и обратно упорядоченных массивах.

5. Задание Г5: сортировка слиянием устойчива, быстрая — нет. Для многоуровневой сортировки отчётов следует использовать устойчивый алгоритм (слияние или sorted()).

6. Встроенная sorted() работает в 7–12 раз быстрее собственных реализаций благодаря реализации на C и гибридному Timsort. В прикладных системах рекомендуется использовать стандартную библиотеку.

11. Контрольные вопросы

1. Три этапа «разделяй и властвуй» на примере слияния.

   Разделение — деление массива пополам (или на блоки width). Властвование — рекурсивная/итеративная сортировка частей. Комбинирование — процедура MERGE.

2. Рекуррентность сортировки слиянием.

   $T(n) = 2\,T(n/2) + \Theta(n)$. Дерево рекурсии: $\log$ n уровней, на каждом $\Theta(n) \rightarrow \Theta(n \log n)$.

3. Нижняя оценка $\Omega(n \log n)$.

   Дерево решений должно иметь $\geq n!$ листьев, высота $\geq \log (n!) = \Theta(n \log n)$.

4. Худшие данные для опорного «первый».

   Уже упорядоченный или обратно упорядоченный массив: разбиения $0 : n-1$, $T(n) = \Theta(n^2)$.

5. Ломуто и Хоар.

   Ломуто — один указатель, опорный в конце, больше обменов. Хоар — два указателя навстречу, меньше обменов, быстрее на практике.

6. Рандомизация опорного.

   Ожидаемое время $\Theta(n \log n)$ для любого входа; вероятность худшего случая ничтожна.

7. Почему быстрая часто быстрее слияния.

   Меньшая константа, работа на месте, лучшая локальность кэша.

8. Глубина рекурсии $O(\log n)$.

   Рекурсивно обрабатывать меньшую часть, большую — в цикле while.

9. Устойчивость.

   Слияние: сравнение «≤» не меняет порядок равных. Быстрая: обмены нарушают порядок.

10. Timsort.

    Выделяет уже упорядоченные runs и сливает их; для коротких — вставками. Адаптивен и устойчив.

12. Рекомендуемая литература

1. Кормен Т. и др. Алгоритмы: построение и анализ. — 3-е изд. — М.: Вильямс, 2013.
2. Седжвик Р., Уэйн К. Алгоритмы на Java. — 4-е изд. — М.: Вильямс, 2013.
3. Hoare C. A. R. Quicksort // The Computer Journal. — 1962. — Vol. 5, No. 1.
4. Кнут Д. Э. Искусство программирования. Т. 3. — 2-е изд. — М.: Вильямс, 2017.
5. Бхаргава А. Грокаем алгоритмы. — 2-е изд. — СПб.: Питер, 2024.
6. Peters T. Timsort description — https://github.com/python/cpython/blob/main/Objects/listsort.txt